Migration guide: SQL Server to Azure SQL Managed Instance

Article • 01/14/2023

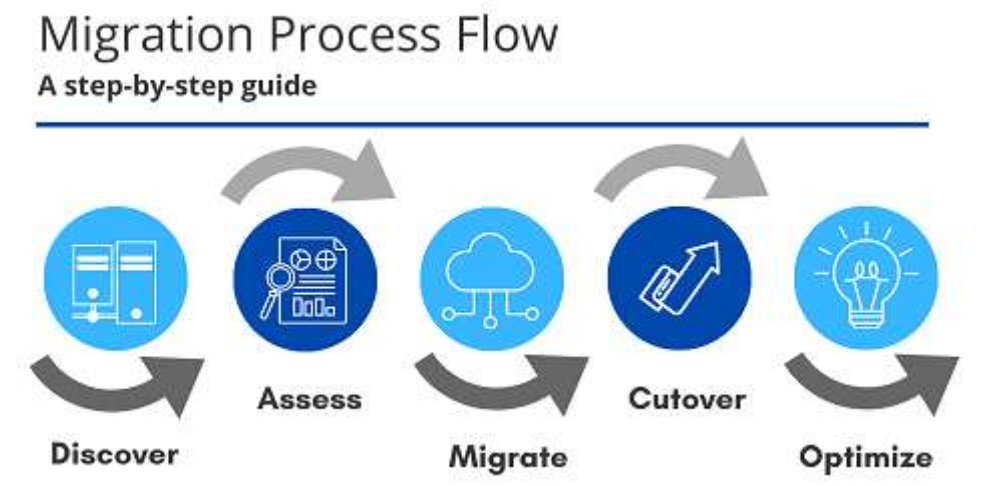
Applies to: Azure SQL Managed Instance

This guide helps you migrate your SQL Server instance to Azure SQL Managed Instance.

You can migrate SQL Server running on-premises or on:

- SQL Server on Virtual Machines
- Amazon EC2 (Elastic Compute Cloud)
- Amazon RDS (Relational Database Service) for SQL Server
- Google Compute Engine
- Cloud SQL for SQL Server - GCP (Google Cloud Platform)

For more migration information, see the migration overview. For other migration guides, see Database Migration.



Prerequisites

To migrate your SQL Server to Azure SQL Managed Instance, make sure you have:

- Chosen a migration method and the corresponding tools for your method.
- Install the Azure SQL migration extension for Azure Data Studio.
- Created a target Azure SQL Managed Instance
- Configured connectivity and proper permissions to access both source and target.

- Reviewed the SQL Server database engine features available in Azure SQL Managed Instance.

Pre-migration

After you've verified that your source environment is supported, start with the pre-migration stage. Discover all of the existing data sources, assess migration feasibility, and identify any blocking issues that might prevent your migration.

Discover

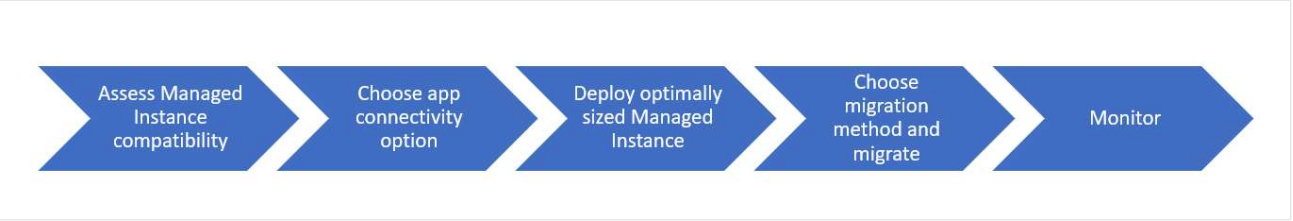
In the Discover phase, scan the network to identify all SQL Server instances and features used by your organization.

Use Azure Migrate to assess migration suitability of on-premises servers, perform performance-based sizing, and provide cost estimations for running them in Azure.

Alternatively, use the Microsoft Assessment and Planning Toolkit (the "MAP Toolkit") to assess your current IT infrastructure. The toolkit provides a powerful inventory, assessment, and reporting tool to simplify the migration planning process.

For more information about tools available to use for the Discover phase, see Services and tools available for data migration scenarios.

After data sources have been discovered, assess any on-premises SQL Server instance(s) that can be migrated to Azure SQL Managed Instance to identify migration blockers or compatibility issues. Proceed to the following steps to assess and migrate databases to Azure SQL Managed Instance:



- Assess SQL Managed Instance compatibility where you should ensure that there are no blocking issues that can prevent your migrations. This step also includes creation of a performance baseline to determine resource usage on your source SQL Server instance. This step is needed if you want to deploy a properly sized managed instance and verify that performance after migration isn't affected.

- Choose app connectivity options.
- Deploy to an optimally sized managed instance where you'll choose technical characteristics (number of vCores, amount of memory) and performance tier (Business Critical, General Purpose) of your managed instance.
- Select migration method and migrate where you migrate your databases using offline migration or online migration options.
- Monitor and remediate applications to ensure that you have expected performance.

Assess

> **Note**
>
> If you are assessing the entire SQL Server data estate at scale on VMware, use **Azure Migrate** to get Azure SQL deployment recommendations, target sizing, and monthly estimates.

Determine whether SQL Managed Instance is compatible with the database requirements of your application. SQL Managed Instance is designed to provide easy lift and shift migration for most existing applications that use SQL Server. However, you may sometimes require features or capabilities that aren't yet supported and the cost of implementing a workaround is too high.

The Azure SQL migration extension for Azure Data Studio provides a seamless wizard based experience to assess, get Azure recommendations and migrate your SQL Server databases on-premises to SQL Server on Azure Virtual Machines. Besides, highlighting any migration blockers or warnings, the extension also includes an option for Azure recommendations to collect your databases' performance data to recommend a right-sized Azure SQL Managed Instance to meet the performance needs of your workload (with the least price).

You can use the Azure SQL Migration extension for Azure Data Studio to assess databases to get:

- Migration readiness - Assessment rules
- Azure right-sized recommendations

To assess your environment using the Azure SQL Migration extension, follow these steps:

1. Open the Azure SQL Migration extension for Azure Data Studio.

2. Connect to your source SQL Server instance
3. Click the *Migrate to Azure SQL* button, in the Azure SQL Migration wizard in Azure Data Studio
4. Select databases for assessment, then click on next
5. Select your Azure SQL target, in this case, Azure SQL Managed Instance
6. Click on *View/Select* to review the assessment report
7. Look for migration blocking and feature parity issues. The assessment report can also be exported to a file that can be shared with other teams or personnel in your organization.
8. Determine the database compatibility level that minimizes post-migration efforts.

To get an Azure recommendation using the Azure SQL Migration extension, follow these steps:

1. Open the Azure SQL Migration extension for Azure Data Studio.
2. Connect to your source SQL Server instance
3. Click the *Migrate to Azure SQL* button, in the Azure SQL Migration wizard in Azure Data Studio
4. Select databases for assessment, then click on next
5. Select your Azure SQL target, in this case, Azure SQL Managed Instance
6. Navigate to the Azure recommendations sections, click on *Get Azure recommendation*
7. Select Collect performance data now. Select a folder on your local computer to store the performance logs, and then select Start.
8. After 10 minutes, Azure Data Studio indicates that a recommendation is available for Azure SQL Managed Instance.
9. Check the Azure SQL Managed Instance card, in the Azure SQL target panel to review your Azure SQL Managed Instance SKU recommendation

To learn more, see Tutorial: Migrate SQL Server to Azure SQL Managed Instance online by using Azure Data Studio.

To learn more, see Tutorial: Migrate SQL Server to Azure SQL Managed Instance offline by using Azure Data Studio.

If the assessment encounters multiple blockers to confirm that your database is not ready for an Azure SQL Managed Instance, then alternatively consider:

- SQL Server on Azure Virtual Machines if both SQL Database and SQL Managed Instance fail to be suitable targets.

Scaled assessments and analysis

The Azure SQL Migration extension for Azure Data Studio and Azure Migrate supports performing scaled assessments and consolidation of the assessment reports for analysis.

If you have multiple servers and databases that need to be assessed and analyzed at scale to provide a wider view of the data estate, see the following links to learn more:

- Migrate databases at scale using automation (Preview) - Azure SQL Migration extension
- Performing scaled assessments using PowerShell - Azure Migrate
- Analyzing assessment reports using Power BI - Azure Migrate

> **Important**
>
> Running assessments at scale for multiple databases can also be automated using **DMA's Command Line Utility** which also allows the results to be uploaded to **Azure Migrate** for further analysis and target readiness.

Deploy to an optimally sized managed instance

You can use the Azure SQL migration extension for Azure Data Studio to get right-sized Azure SQL Managed Instance recommendation. The extension collects performance data from your source SQL Server instance to provide right-sized Azure recommendation that meets your workload's performance needs with minimal cost. To learn more, see Get right-sized Azure recommendation for your on-premises SQL Server database(s)

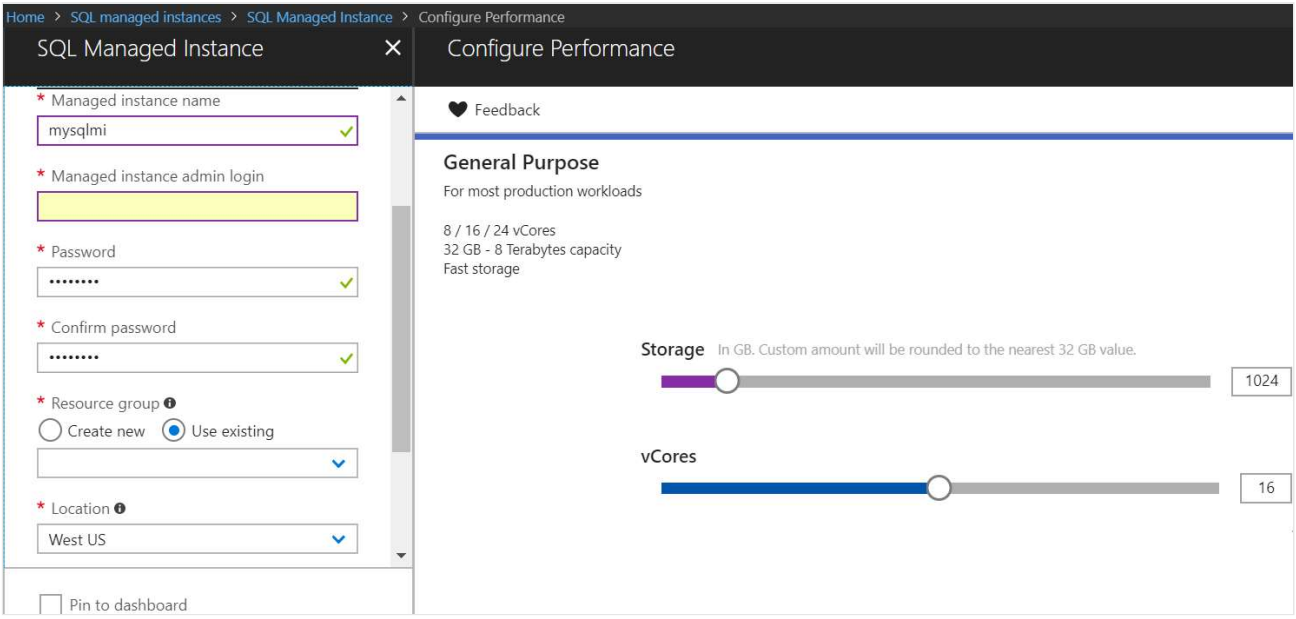
Based on the information in the discover and assess phase, create an appropriately sized target SQL Managed Instance. You can do so by using the Azure portal, PowerShell, or an Azure Resource Manager (ARM) Template.

SQL Managed Instance is tailored for on-premises workloads that are planning to move to the cloud. It introduces a purchasing model that provides greater flexibility in selecting the right level of resources for your workloads. In the on-premises world, you're probably accustomed to sizing these workloads by using physical cores and IO bandwidth. The purchasing model for managed instance is based upon virtual cores, or "vCores," with additional storage and IO available separately. The vCore model is a simpler way to understand your compute requirements in the cloud versus what you use on-premises today. This purchasing model enables you to right-size your destination environment in the

cloud. Some general guidelines that might help you to choose the right service tier and characteristics are described here:

- Based on the baseline CPU usage, you can provision a managed instance that matches the number of cores that you're using on SQL Server, having in mind that CPU characteristics might need to be scaled to match VM characteristics where the managed instance is installed.
- Based on the baseline memory usage, choose the service tier that has matching memory. The amount of memory can't be directly chosen, so you would need to select the managed instance with the amount of vCores that has matching memory (for example, 5.1 GB/vCore in standard-series (Gen5)).
- Based on the baseline IO latency of the file subsystem, choose between the General Purpose (latency greater than 5 ms) and Business Critical (latency less than 3 ms) service tiers.
- Based on baseline throughput, pre-allocate the size of data or log files to get expected IO performance.

You can choose compute and storage resources at deployment time and then change it afterward without introducing downtime for your application using the Azure portal:



To learn how to create the VNet infrastructure and a managed instance, see Create a managed instance.

> **Important**

It is important to keep your destination VNet and subnet in accordance with **managed instance VNet requirements**. Any incompatibility can prevent you from creating new instances or using those that you already created. Learn more about **creating new** and **configuring existing** networks.

Migrate

After you have completed tasks associated with the Pre-migration stage, you're ready to perform the schema and data migration.

Migrate your data using your chosen migration method.

SQL Managed Instance targets user scenarios requiring mass database migration from on-premises or Azure VM database implementations. They are the optimal choice when you need to lift and shift the back end of the applications that regularly use instance level and/or cross-database functionalities. If this is your scenario, you can move an entire instance to a corresponding environment in Azure without the need to rearchitect your applications.

To move SQL instances, you need to plan carefully:

- The migration of all databases that need to be collocated (ones running on the same instance).
- The migration of instance-level objects that your application depends on, including logins, credentials, SQL Agent jobs and operators, and server-level triggers.

SQL Managed Instance is a managed service that allows you to delegate some of the regular DBA activities to the platform as they're built in. Therefore, some instance-level data doesn't need to be migrated, such as maintenance jobs for regular backups or Always On configuration, as high availability is built in.

This article covers two of the recommended migration options:

- Azure SQL migration extension for Azure Data Studio - migration with near-zero downtime.
- Native `RESTORE DATABASE FROM URL` - uses native backups from SQL Server and requires some downtime.

This guide describes the two most popular options - Azure Database Migration Service (DMS) and native backup and restore.

For other migration tools, see Compare migration options.

Migrate using the Azure SQL migration extension for Azure Data Studio (minimal downtime)

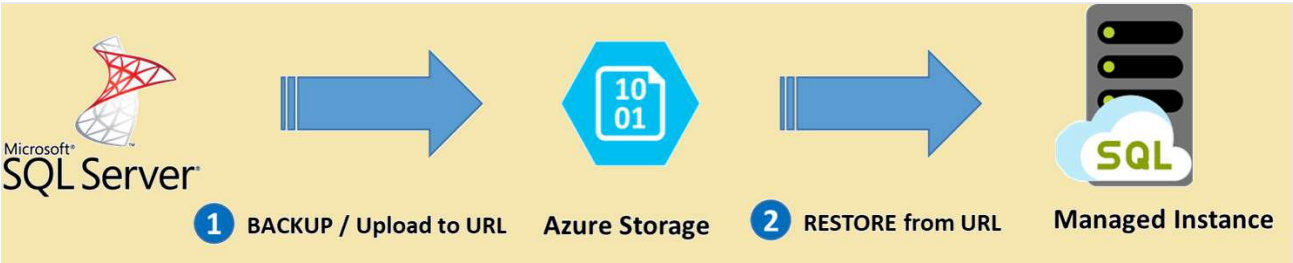
To perform a minimal downtime migration using Azure Data Studio, follow the high-level steps below. For a detailed step-by-step tutorial, see Migrate SQL Server to an Azure SQL Managed Instance online using Azure Data Studio:

1. Download and install Azure Data Studio and the Azure SQL migration extension.
2. Launch the Migrate to Azure SQL Migration wizard in the extension in Azure Data Studio.
3. Select databases for assessment and view migration readiness or issues (if any). Additionally, collect performance data and get right-sized Azure recommendation.
4. Select your Azure account and your target Azure SQL Managed Instance from your subscription.
5. Select the location of your database backups. Your database backups can either be located on an on-premises network share or in Azure Blob Storage container.
6. Create a new Azure Database Migration Service using the wizard in Azure Data Studio. If you've previously created an Azure Database Migration Service using Azure Data Studio, you can reuse the same if desired.
7. *Optional*: If your backups are on an on-premises network share, download and install self-hosted integration runtime on a machine that can connect to the source SQL Server, and the location containing the backup files.
8. Start the database migration and monitor the progress in Azure Data Studio. You can also monitor the progress under the Azure Database Migration Service resource in Azure portal.
9. Complete the cutover.
    a. Stop all incoming transactions to the source database.
    b. Make application configuration changes to point to the target database in Azure SQL Managed Instance.
    c. Take any tail log backups for the source database in the backup location specified.
    d. Ensure all database backups have the status Restored in the monitoring details page.
    e. Select Complete cutover in the monitoring details page.

Backup and restore

One of the key capabilities of Azure SQL Managed Instance to enable quick and easy database migration is the native restore of database backup (`.bak`) files stored on Azure Storage . Backing up and restoring are asynchronous operations based on the size of your database.

The following diagram provides a high-level overview of the process:



> **Note**
>
> The time to take the backup, upload it to Azure storage, and perform a native restore operation to Azure SQL Managed Instance is based on the size of the database. Factor a sufficient downtime to accommodate the operation for large databases.

The following table provides more information regarding the methods you can use depending on source SQL Server version you're running:

Expand table

Step	SQL Engine and version	Backup/restore method
Put backup to Azure Storage	Prior to 2012 SP1 CU2	Upload .bak file directly to Azure Storage
	2012 SP1 CU2 - 2016	Direct backup using deprecated WITH CREDENTIAL syntax
	2016 and above	Direct backup using WITH SAS CREDENTIAL
Restore from Azure Storage to a managed instance		RESTORE FROM URL with SAS CREDENTIAL

> **Important**

- When you're migrating a database protected by Transparent Data Encryption to a managed instance using native restore option, the corresponding certificate from the on-premises or Azure VM SQL Server needs to be migrated before database restore. For detailed steps, see Migrate a TDE cert to a managed instance.
- Restore of system databases is not supported. To migrate instance-level objects (stored in `master` or `msdb` databases), we recommend to script them out and run T-SQL scripts on the destination instance.

To migrate using backup and restore, follow these steps:

1. Back up your database to Azure Blob Storage. For example, use backup to url in SQL Server Management Studio. Use the Microsoft Azure Tool to support databases earlier than SQL Server 2012 SP1 CU2.

2. Connect to your Azure SQL Managed Instance using SQL Server Management Studio.

3. Create a credential using a Shared Access Signature to access your Azure Blob storage account with your database backups. For example:

```
SQL

CREATE CREDENTIAL [https://mitutorials.blob.core.windows.net/databases]
WITH IDENTITY = 'SHARED ACCESS SIGNATURE'
, SECRET = 'sv=2017-11-09&ss=bfqt&srt=sco&sp=rwdlacup&se=2028-09-
06T02:52:55Z&st=2018-09-
04T18:52:55Z&spr=https&sig=WOTiM%2FS4GVF%2FEEs9DGQR9Im0W%2BwndxW2CQ7%2B5fH
d7Is%3D'
```

4. Restore the backup from the Azure storage blob container. For example:

```
SQL

RESTORE DATABASE [TargetDatabaseName] FROM URL =
  'https://mitutorials.blob.core.windows.net/databases/WideWorldImporters-
Standard.bak'
```

5. Once restore completes, view the database in **Object Explorer** within SQL Server Management Studio.

To learn more about this migration option, see Restore a database to Azure SQL Managed Instance with SSMS.

> **Note**
>
> A database restore operation is asynchronous and can be retried. You might get an error in SQL Server Management Studio if the connection breaks or a time-out expires. Azure SQL Database will keep trying to restore database in the background, and you can track the progress of the restore using the sys.dm_exec_requests and sys.dm_operation_status views.

Data sync and cutover

When using migration options that continuously replicate / sync data changes from source to the target, the source data and schema can change and drift from the target. During data sync, ensure that all changes on the source are captured and applied to the target during the migration process.

After you verify that data is the same on both source and target, you can cut over from the source to the target environment. It's important to plan the cutover process with business / application teams to ensure minimal interruption during cutover doesn't affect business continuity.

> **Important**
>
> For details on the specific steps associated with performing a cutover as part of migrations using DMS, see Performing migration cutover.

Post-migration

After you've successfully completed the migration stage, go through a series of post-migration tasks to ensure that everything is functioning smoothly and efficiently.

The post-migration phase is crucial for reconciling any data accuracy issues and verifying completeness, and addressing performance issues with the workload.

Monitor and remediate applications

Once you've completed the migration to a managed instance, you should track the application behavior and performance of your workload. This process includes the following activities:

- Compare performance of the workload running on the managed instance with the performance baseline that you created on the source SQL Server instance.
- Continuously monitor performance of your workload to identify potential issues and improvement.

Perform tests

The test approach for database migration consists of the following activities:

1. **Develop validation tests**: To test database migration, you need to use SQL queries. You must create the validation queries to run against both the source and the target databases. Your validation queries should cover the scope you've defined.
2. **Set up test environment**: The test environment should contain a copy of the source database and the target database. Be sure to isolate the test environment.
3. **Run validation tests**: Run the validation tests against the source and the target, and then analyze the results.
4. **Run performance tests**: Run performance test against the source and the target, and then analyze and compare the results.

Use advanced features

You can take advantage of the advanced cloud-based features offered by SQL Managed Instance, such as built-in high availability, threat detection, and monitoring and tuning your workload.

Azure SQL Analytics allows you to monitor a large set of managed instances in a centralized manner.

Some SQL Server features are only available once the database compatibility level is changed to the latest compatibility level (150).

Next steps

- See Service and tools for data migration for a matrix of the Microsoft and third-party services and tools that are available to assist you with various database and data migration scenarios as well as specialty tasks.
- To learn more about the Azure SQL Migration extension see:
  - Migrate databases with Azure SQL Migration extension for Azure Data Studio
  - Tutorial: Migrate SQL Server to Azure SQL Managed Instance online by using Azure Data Studio.
  - Tutorial: Migrate SQL Server to Azure SQL Managed Instance offline by using Azure Data Studio.
- To learn more about Azure SQL Managed Instance see:
  - Service Tiers in Azure SQL Managed Instance
  - Differences between SQL Server and Azure SQL Managed Instance
  - Azure total Cost of Ownership Calculator
- To learn more about the framework and adoption cycle for Cloud migrations, see
  - Cloud Adoption Framework for Azure
  - Best practices for costing and sizing workloads migrate to Azure
- To assess the Application access layer, see Data Access Migration Toolkit (Preview)
- For details on how to perform Data Access Layer A/B testing see Database Experimentation Assistant.

Feedback

Was this page helpful? Yes No

Provide product feedback